

Ph.D. Research Exam Report

Parallel GA for TSP and MAXSAT

Swetha Varadarajan
swesri@cs.colostate.edu

June 4, 2019

Abstract

Genetic algorithms are embarrassingly parallel in nature, but the standard genetic algorithms suffer from several constraints such as (a) memory required per genome being significantly large (b) communication cost between generations exceeding the computation cost and (c) sequential genetic operators that hinder their effective implementation on parallel architectures. Parallel Genetic Algorithms (PGA) improve the solution quality as well as reduces the execution time. This article is a survey on existing parallel genetic algorithms for two NP-Hard problems, namely the Travelling Salesman Problem (TSP) and Maximal Satisfiability (MAXSAT) on different computer architectures - CPU, GPU, FPGA. This article also discusses recent studies on the portfolio method of parallelization and hybrid metaheuristics on heterogeneous systems. From this study, we can infer that there have been very few attempts in parallelization of genetic algorithms for TSP and more specifically for MAXSAT. Although there exists a powerful CPU based PGA for TSP, this technique is challenging to implement on the GPUs. Therefore, there is a need to research on the design and implementation of effective genetic operators and algorithms for parallel architecture.

Note: The keywords used for the literature search were parallel, MAXSAT, TSP, genetic, GPU. I missed out searching for **distributed** genetic algorithms. However, I plan to include them during my talk. While there is only one article on distributed MAXSAT genetic algorithm, there are several TSP GA solvers constructed on a distributed environment.

Contents

1	Introduction	3
1.1	Preliminaries	3
1.2	Exam Overview	4
2	Parallel Genetic Algorithms for Travelling Salesman Problem	5
2.1	Review of initial papers	5
2.1.1	Mapping the global structures of TSP fitness landscapes	5
2.1.2	Leveraging TSP solver Complementarity through Machine Learning	7
2.2	PGA on CPU	8
2.3	PGA on GPU	10
2.4	PGA on FPGA	10
2.5	Summary	12
3	Parallel Genetic Algorithms for Maximal Satisfiability	13
3.1	Review of initial papers	13
3.1.1	Adaptive memory based local search for MAXSAT	13
3.1.2	Tunneling Between Plateaus: Partition Crossover	14
3.2	PGA on CPU	14
3.3	PGA on GPU	16
3.4	PGA on FPGA	16
3.5	Summary	17
4	Alternative Parallel Framework	18
4.1	Hybrid Metaheuristics on heterogenous system	18
4.2	Portfolio method	19
5	Conclusion	19

1 Introduction

With the decline of Moore’s law, the proliferation of heterogeneous, exascale computing, GPUs and other accelerators are increasing. Genetic algorithms, inspired by Darwin’s theory of evolution, have been successfully implemented as function optimizers in different fields such as search techniques, machine learning algorithms and are found to be embarrassingly parallel [6] in nature. Parallel Genetic Algorithms(PGA) improve the solution quality as well as reduces the execution time. When dealing with population-based genetic algorithms, it is intuitive to perform genetic operations on each pair of chromosome in one parallel computing unit. However, there are several challenges associated with the effective implementation of PGAs.

First, the memory required per genome can be prohibitively large depending on the genetic operators. Second, is the communication cost before, during, and after a generation. Before a generation could begin, the population is searched either randomly or in a specific pattern to find the appropriate mating partner. During a generation, there may be communication cost associated with the evaluation of the chromosome. At the end of the generation, there can be communication cost depending on how the algorithm chooses to replace the original population. Third challenge is the quality of solution which is directly dependent on the crossover, selection, and mutation operators. Most of the genetic operators in the literature are inherently sequential and do not expose fine-grain parallelism.

There are several PGA models studied in the literature. The most promising ones are the cellular genetic algorithms and the island model. Here, the population is divided into sub-populations, and the diversity is preserved by individuals migrating from one island to another at a specific rate. However, there is a necessity to tune the migration rates while bearing the communication cost associated with the migration. In this report, we analyze the challenges associated with the design of PGAs for two NP-hard problems, namely, Travelling Salesman Problem (TSP) and Maximal Satisfiability (MAXSAT) on three different computer architectures namely CPU, GPU, FPGA. Section 1.1 gives some background on the terminologies of the genetic algorithm, and section 1.2 gives an overview of the structure and scope of this article.

1.1 Preliminaries

Algorithm 1 lists the steps in a population-based genetic algorithm. This section reviews some of the terminologies related to a standard genetic algorithm that will be used in the subsequent sections.

Initial Population: Represents the set of randomly generated solution encoded in a string called chromosome. For TSP, this string represents the tour, and for MAXSAT, this string is a bunch of 0s and 1s corresponding to the truth assignment of the variables.

Evaluation function: Since GAs are used as function optimizers, each chromosome evaluates to a value. For TSP, it is the length of the tour, and for MAXSAT, it is the number of unsatisfied clauses.

Local Search (LS): The success of genetic algorithms depends on the initial population. So, most genetic algorithms are combined with the local search heuristics. LS can either be applied once to

Algorithm 1 Pseudocode of a Canonical Genetic Algorithm

```
1:  $P \leftarrow \text{GenerateInitialPopulation}()$ 
2:  $\text{Evaluate}(P)$ 
3: while not  $\text{Termination Condition}()$  do
4:    $P' \leftarrow \text{SelectParents}(P)$ 
5:    $P' \leftarrow \text{ApplyGeneticOperators}(P')$ 
6:    $\text{Evaluate}(P')$ 
7:    $P \leftarrow \text{SelectNewPopulation}(P, P')$ 
8: end while
9: Return The best solution found;
```

the initial population or can be applied after the recombination process.

Selecting mating partner: Most genetic algorithms employ a random search to pick a partner for recombination. However, depending on the type of computer architecture, the scheme varies. Nevertheless, the communication cost associated with this selection process is something that a PGA designer must consider.

Migration: For specific GA models, population are divided into subpopulations, and the generations evolve locally in each sub-population. However, GA is prone to premature convergence if the population is not mixed correctly and hence, some of the individuals have to be migrated across the subpopulations.

Randomization: To preserve diversity, most GA framework involves some form of randomization. It is hard to generate random numbers in specific computer architectures.

Genetic Operators: There are three basic genetic operators, namely the selection, mutation, and recombination (crossover). The mutation is applied to one single chromosome, whereas recombination is applied to two sets of chromosomes to produce one or multiple children. Selection operator does not change the evaluation function of the chromosome instead determines if a particular individual should be present in the population or not.

Termination: GAs either terminate on reaching the desired optimal value or meeting with specific criteria in terms of the number of generations or execution time.

1.2 Exam Overview

The focus of this report is to do a detailed survey of the existing PGAs for TSP and MAXSAT. Each survey has three parts. First is the general summary of the techniques in the current paper. Emphasis is given to the genetic algorithm details and the test suite. Second is the evaluation of the paper. This helps to identify the gap in the current article. Most of the papers are evaluated based on the problem instance and experimental design choices.

The third part is to establish the relevance of the paper to the design of PGAs. The relevance is determined based on the following five categories. (1) the memory required per genome (2) Com-

munication cost (3) Computation cost and its trade-off with the communication cost. (4) Quality of solution in comparison with the state-of-the-art techniques. (5) If the proposed PGA is worthwhile to implement on the chosen architecture and if the proposed algorithm utilizes the architecture specific resources.

Section 3 reviews the papers on parallel genetic algorithms for MAXSAT whereas section 2 for TSP. In each section, the state-of-the-art architectures for various parallel programming architecture CPU, GPU, and FPGA are discussed. These sections also discuss papers that were provided as part of the exam. Section 4 discusses alternate parallel frameworks such as portfolio and hybrid metaheuristics on heterogeneous systems. Finally, the report ends with open challenges and future directions in Section 5.

2 Parallel Genetic Algorithms for Travelling Salesman Problem

Travelling Salesman Problem is a widely studied NP-hard problem. The goal of TSP is to find the shortest Hamiltonian cycle in a complete weighted graph. The state-of-the-art inexact TSP solver is based on the genetic algorithm. Section 2.2 discusses the parallel implementation of this algorithm and enumerates the shortcomings of the current implementation. Section 2.3 and 2.4 discusses the recent advances in the parallelization of TSP in the GPU and FPGA platform, respectively. Section 2.1 surveys the two papers provided as a part of this exam along with its relevance to the design and challenges and design of PGA.

2.1 Review of initial papers

In this section, we will review two out of the three TSP papers given as a part of the exam. Section 2.2 discusses the details of the third TSP paper [25].

2.1.1 Mapping the global structures of TSP fitness landscapes

In this [26], the authors develop visualization techniques for mapping the global structure of TSP landscape. To construct the global landscape, they use something called a local optima network. This network has the local optima as the nodes, and the edges are called escape edges. These escape edges represent the transition probabilities from one local optima to the next. These probabilities are collected when performing 10,000 iterations of a local search heuristic called chained LK.

After extracting the local optima network (LON), they systematically devise an approach to identify the so-called funnel structures. A funnel is nothing but a region in the TSP landscape where several local optima converge to one single point which need not be the global optima. Figure 1 depicts a simple two funnel structure. There are three types of LON nodes, namely, the attractor, sink, or plateau based on the type of local optima. Attractor nodes are those that have high probabilities on their escape edges. Sink nodes are those that do not have an escape node, and plateau nodes are set

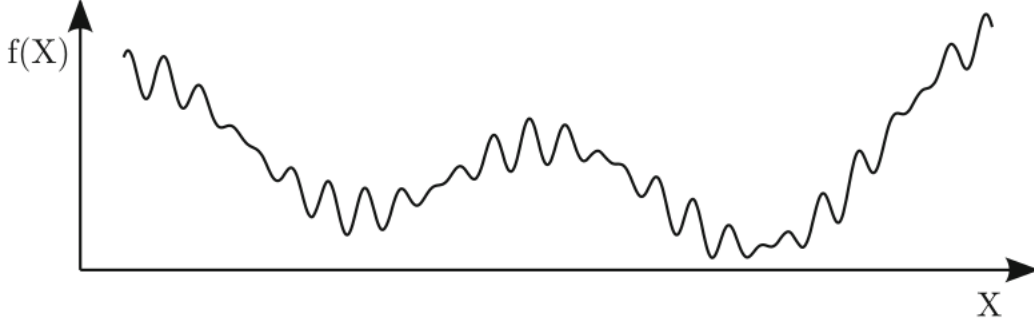


Figure 1: Depiction of two funnels

of nodes with the same objective function value. A plateau node can either be an attractor or the sink node.

The authors experiment on about 20 TSP instances. The 10 of the 20 instances are randomly generated, and the remaining ten are from the TSPLIB dataset. They provide insights on six metrics. First, they provide details on the ten general statistics of the sampled LON across the instance set. Please refer Table 2 in [26]. One key observation is where the network sampled for two instances, u1060 and fl1577 reveal that they have a highly neutral plateau, and each run of the local search produced a different path.

Second, they focus on the 6-8 features characterizing the landscape’s global structure. They conclude their analysis stating that the presence of large plateau at the global optimum level may reflect as an easy to search instance whereas the presence of a large sub-optimal plateau may act as a trap that is difficult to escape. Third, they provide the 2D and 3D visualization of the LON for some critical instance and analyze the fitness landscape in the subsequent section. Specifically, they try to answer one exciting question - whether all the plateaus with equal evaluation found across the different runs are a single large plateau. They use a metric called bond edges, the difference in the common edges to analyze. They conclude that the higher the bond edge metric, the plateaus are disconnected, the lower the value, there might be ways to connect them.

Fifth, they study the effects of the sampling parameters in the landscape metrics and conclude that the sampling methodology used in this study is valid and useful. Finally, they perform a correlation between landscape metrics and heuristic search performance. They obtain a 0.74 on Pearson correlation, revealing that their methodology reflects the searched dynamics of the chosen Local search algorithm. In conclusion, they find out that the *big valley* structure of the landscape is not typical, and in fact, the global structure exhibits something called *multi funnel* landscape.

Evaluation: This article does an in-depth analysis of the several features of the global structure of TSP landscape. Although the TSP instances range in between 150 and 1500 cities, I believe that the metrics defined in this study is highly useful for higher valued instances as well. My only concern

is the bias of the network towards one particular local search metric. It would be interesting to view the LON sampled from multiple Local search techniques.

Relevance: The visualization may be useful in designing appropriate parallel genetic operators by carefully parallelizing the search space. Since the success of genetic algorithms depends on the initial population, this visualization analysis can help to determine the proper local search front-end.

2.1.2 Leveraging TSP solver Complementarity through Machine Learning

In this paper [18], the authors develop a machine learning based algorithm selector to choose the best among five state-of-the-art TSP solvers for a given TSP instance. The five TSP solvers are two versions of Lin-Kernighan heuristics (LKH), two versions of Edge Assembly Crossover (EAX) and Multiagent Optimization System (MAOS). Section 2.2 details the EAX implementation, the state-of-the-art genetic algorithm based solver for TSP.

LKH [15] is the Helsgun’s variant of the Lin-Kernighan heuristic. LKH is the single best TSP solver in the past decade or two. The LK heuristic is the generalized version of the 2opt and 3opt local search. It is a variable depth search method with adaptive edge exchanges. The LKH heuristic involves five or more edge exchanges.

MAOS [34] is a nature-inspired method where several agents have only limited knowledge of the TSP instance and explore possible solutions in parallel and cooperation with each other. In a simplified MAOS version, each agent manipulates on minimal declarative knowledge. They adopt similar environment rules for TSP and also include some simple and efficient components, including two improving heuristics based on a generalized edge assembly recombination. It has observed that MAOS exceeds the performance of LKH and EAX for certain problem instances.

They use a total of 405 features from five feature, namely - TSPmeta, UBC and Pihera. They use a total of 1845 instances collected from a wide variety of dataset - RUE instances, TSPLIB instances, National instances, VLSI instances, Netgen and morphed instances.

PAR10 score [2], which averages the running time over a given instance, counting runs that exceed the given cutoff time as ten times the cutoff is used to measure the performance. They evaluated the algorithm selector based on three supervised learning technique, namely classification, regression, and paired regression. The classification model labels each instance with the best performing TSP solver. The regression model predicts the runtime of each solver on an instance and chooses the solver with the lowest predicted runtime. The pairwise regression model predicts the performance difference between two solvers for each pair of solvers, and they select the solver with the best-predicted performance difference. They construct a total of 70 machine learning algorithm selectors using a combination of classification and regression trees, support vector machines, two variants of random forests and MARS models. Together with three feature selection methods (sffs, sfbs, (10+5) genetic algorithm), they develop a total of 210 algorithm selectors.

They compare the results against two other solvers namely - virtual best solver and single-best solver. Overall, they found the variant of EAX as the single best solver, which is a genetic algorithm

based solver. Furthermore, they found that the algorithm selector based on SVM performs the best among the 210 selectors.

Evaluation: Overall, this research work does excellent work in finding the single best TSP solver. Since they are using the running time as a performance measure, I believe the results will vary if (1) they consider the parallel versions (2) appropriate machine configurations. Furthermore, the article does not discuss the overhead of machine learning techniques and the practical use of the algorithm selector.

Relevance: It is essential to note that the genetic algorithm based EAX algorithm is found to be the single best TSP solver. Examining the feature set may provide valuable insights into the design of parallel genetic algorithms for TSP.

2.2 PGA on CPU

Edge Assembly Crossover (EAX) [25] is the state-of-the-art inexact TSP solver based on genetic algorithms. In this section, we will review the parallelized version of EAX [16] in detail.

The EAX algorithm follows the standard canonical genetic Algorithm. However, it uses several sophisticated mechanisms to maintain a balance between exploitation and exploration of the population. To ensure exploitation on a given set of parents, it does something called brood selection, wherein, it generates multiple children instead of one and replaces the first parent with the best child.

After creating the initial population using the greedy local search algorithm with the 2-opt neighborhood, the authors describe five significant steps in the algorithm, as seen in Figure 2. The first step is the selection of a mating partner through randomization. They divide the population into sub-populations equal to the number of parallel units(nodes) called chunks. They send the chunks and frequency table to the respective worker nodes in step 2. The third step is where the offsprings are generated, and the best offspring replaces the first parent. These replaced offspring, along with the updated frequency table, is sent back to the master node, and the global frequency table is finally updated in the fifth stage.

Although the parallelization is straight-forward, the advantage of this implementation is its ability to find high-quality solutions for up to 180,000 city problems in less than a day. They were able to find new optimal solutions for two problem instances.

Evaluation: Following are some of the gaps in the literature. First, the parallel EAX algorithm makes use of additional memory to preserve diversity and thereby produce high-quality results without getting stuck at a local optimum. There is no description of the memory used in the algorithm. From inspection of the publicly available EAX code, it consumes about $90N$ ints memory per chromosome, where N is the number of cities. This memory is prohibitively high for 100,000 city problems.

Second, the proposed algorithm has been tested only on the art TSP instances. There is no mention of how the proposed algorithm works for smaller cases. Probably, the communication cost would exceed the computation cost, and therefore, the parallel version is not efficient for smaller TSP instances.

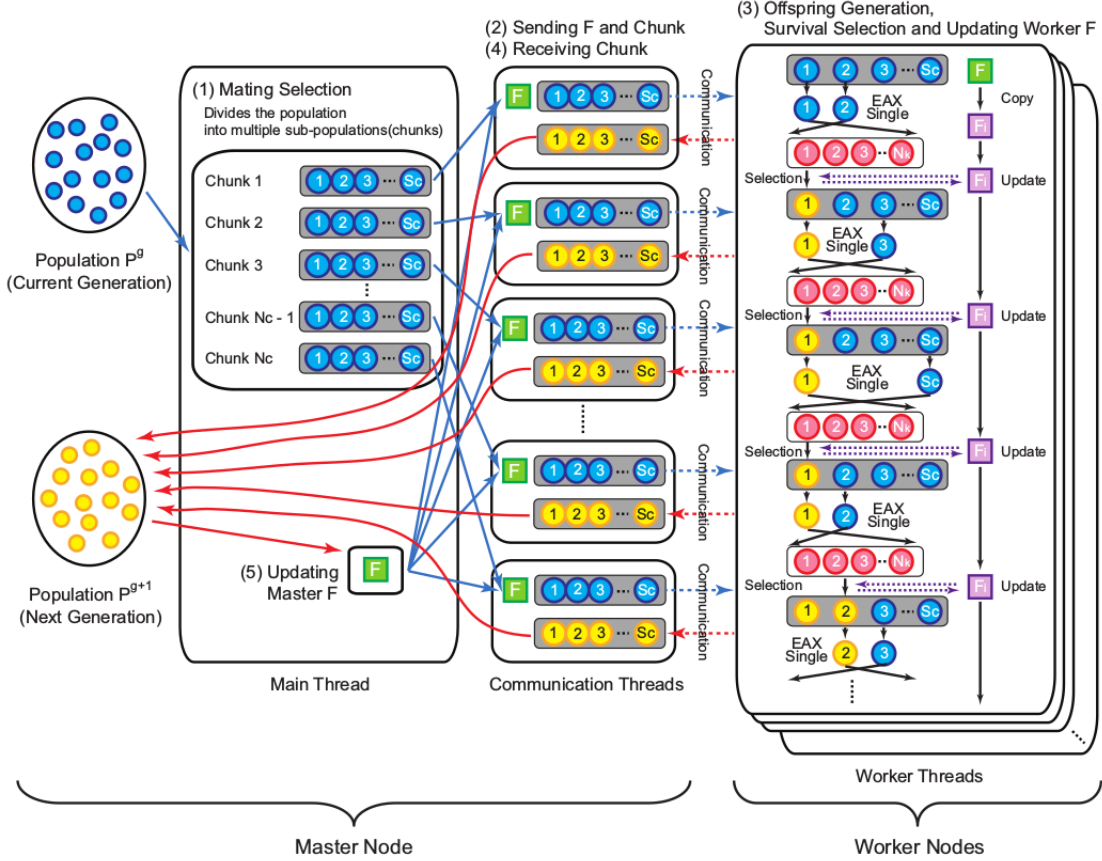


Figure 2: Diagram of the proposed parallel GA [16] based on the master/worker model. Blue and yellow circles are the population members of the current and next generations, respectively. Red circles are offspring solutions generated by EAX-Single. Green and purple squares are the master and each worker edge frequency tables, respectively.

Third, the authors do not report the performance results of the initial 2opt local search. The time it took to generate the initial population using 2opt method is not provided. Furthermore, the solution quality of the initial population created by 2opt is also not reported. This observation is essential to understand the efficiency of the proposed algorithm.

Finally, from the original EAX paper [25], the authors state that the fitness landscapes of TSP have a typical big valley structure. Their algorithm also spends significant time in the termination process by considering this observation from [3]. However, in [26], the authors find the existence of multiple valley funnels. Although there are only ten common test cases between [25] and [26], there is a potential to explore this aspect to reduce the computational complexity of this highly sophisticated termination process.

2.3 PGA on GPU

In this section, a survey of three GPU implementations [11, 36, 17] is discussed. The evaluation and relevance for all the three methods are summarized together at the end. All three methods exhibit high similarity in the design and implementation of the PGA on GPU. Table 1 contrasts and summarizes the techniques, experientnal setup and results from these three papers.

Evaluation: Comparing the three papers, paper3 [17] seems to have the best implementation in terms of solution quality, problem size, and the usage of parallel genetic operators. Despite several challenges [5] in implementing irregular applications like TSP on GPUs, paper3 seems to be perfect. Four shortcoming of the paper3 are (1) the lack of comparison against LKH and EAX methods (2) Small dataset for evaluation (3) Details of the memory mangement is not provided. (4) Details on the calculation of the tour length is not provided.

2.4 PGA on FPGA

In [12], the authors implement the steady-state genetic algorithm for TSP on multiple FPGA platforms. A steady-state genetic algorithm is one in which the population remains constant, but not all the individuals in the population participate in a single generation. For a given set of parents, the offspring are generated and inserted back into the population, and hence, this steady-state GA requires fewer memory requirements as well as reduced communication costs. This method is one of the first versions where the PGA is implemented on multiple FPGA system and has a runtime control on the migration rate of the algorithm.

The proposed system explores two levels of parallelism. The fine-grained is achieved in terms of Selection-Crossover-Mutation/Evaluate (SCM/E) unit local to one FPGA, and the coarse grain is through multiple FPGAs. The users can control the migration rate and frequency between the FPGAs during the runtime.

Four Maseler MAX3A Vertis acceleration cards each with Xilinx Virtex-6 SX475T FPGA is used for the experimental evaluation. They test the approach on a 64 city problem using a population of 32. they evaluate three different applications, including TSP and provide some general conclusions.

Table 1: Summary of PGA features of three GPU implementations for TSP

Features	Paper1 [11]	Paper2 [36]	Paper3 [17]
Year	2011	2014	2016
Programming language	CUDA	OpenCL	CUDA
Machine used	GeForce GTX285	AMD Radeon HD5970	GTX 980
Thread management	Group of threads mapped to one individual	Each thread mapped to each individual in the population	Group of threads mapped to one individual
Memory management	25 registers per thread and $(20n + 28)$ bytes shared memory per thread block where n is the number of cities.	Population and evaluation function stored in Global memory	NA
Local search	NA	NA	2opt
Population initialization	Randomized	Randomized	Randomized
Selection for mating	NA	Roulette-wheel	Random
Crossover operator	Parallel OX	PMX if the crossover probability is greater than a threshold. If not, the parents are copied to the children.	Parallel BCSCX, ARX, hybrid of the two
Mutation operator	2opt local search	Mutation happens if the probability of mutation is above a certain threshold.	NA
Test Dataset	TSPLIB	TSPLIB	7 samples
Max. number of cities	500	2392	100,000
Population size	60	200	Varies
Results - Execution time	A max. of $24.2\times$ fast compared to CPU	Speed-up with respect to the CPU version: Proportional to number of cities	A max of $20\times$ fast compared to CPU
Results - Solution quality	None reaches global	NA	Able to obtain the best known tour for 100,000 city problem.

Work	Year	# of FPGAs	Runtime Param.	Migration Param.	Parallel SCM/E	GA Type	Platform
[11]	1999	Multiple	N/A	No	No	Steady-State	SFL
[12]	2000	Single	N/A	N/A	No	Steady-State	PCIGEN10K
[20]	2005	Multiple	N/A	No	No	Generational	Spartan II-E
[15]	2006	Single	N/A	N/A	No	Generational	Stratix EPS1S10
[10]	2006	Single	N/A	N/A	No	Generational	Cyclone FPGA
[14]	2012	Single	N/A	N/A	No	Generational	Virtex 4 (LX25)
[17]	2013	Single	N/A	No	No	Generational	Virtex 6 (LX240T-1)
[16]	2014	Single	SCM rates	Run-time	Yes	Generational	Virtex 6 (SX475T)
Proposed	2015	Multiple	SCM rates	Run-time	Yes	Steady-State	Virtex 6 (SX475T)

Figure 3: Qualitative Comparisons of Multiple FPGA-based PGAs. Please refer [12] for the appropriate article in the first column

Table 2: List of TSP Genetic operators surveyed

Crossover	Mutation	Local search	Selection
ARX [17]	Flip-flop [12]	Chained LK [26]	Tournament [12]
BCSCX [17]	Exchange [12]	LKH [18]	Roulette-wheel [36]
EAX [25]	Constraint [12]	2opt [26, 25]	
N-point [12]			
Blending [12]			
SCX [17]			
OX [11]			
PMX [36]			

One insightful observation on TSP is that the current algorithm consumes much memory and can be further optimized to fit 2 SCM/E units per FPGA.

Evaluation: The figure 3 taken from the paper shows a brief history in the design of FPGA based TSP solvers since 1999. Compared to the state-of-the-art CPU techniques, this method is very primitive but opens up several future directions to explore the design of PGA for TSP using FPGAs.

Relevance: The relevance of this article to the design of PGAs is straightforward.

2.5 Summary

To summarize the PGA methods for TSP solvers, EAX stands out as the single best genetic algorithm based solver for TSP on the CPU platform. There are very few attempts concerning FPGAs. With regards to GPUs, one specific approach based on ARX and BCSCX crossover claim to find the global optima on 100,000 city problem. Table 2 summarizes the genetic operators surveyed in this section. Some of the open challenges in the design of PGAs are

- **Design of parallel genetic operators:** BCSCX, ARX and OX are some of the parallel genetic operators surveyed. The future genetic operators or the enhancements to the existing methods

must incorporate the parallel design choices.

- **Communication effective evaluation function:** For TSP, there is a need to calculate the tour length, which involves a lookup of edges from memory. This adds overhead on the communication cost, and hence, there is a need to design a communication effective evaluation.
- **Lightweight crossover operators:** EAX is a “heavy” operator in terms of the amount of space it occupies in the memory. For increasing the resource utilization of parallel architectures, there is a need to design lightweight crossover operators.

3 Parallel Genetic Algorithms for Maximal Satisfiability

Maximal Satisfiability is a widely studied NP-hard problem. It is the optimization of SAT problem wherein, the objective is to find an assignment to the variables in a CNF that satisfies a given set of clauses. The goal of the optimization problem is to minimize the number of unsatisfied clauses. MAXSAT is useful in several applications such as circuit designing, graph coloring, planning etc.

There are several local search sequential and their parallel versions [14] to solve the MAXSAT efficiently. But, local search methods suffer from two serious flaws [8]: One is that local search solvers get stuck in local minima and takes a significant amount of time to find further improving moves and two, they have poor performance on application SAT instances.

Genetic algorithms are one approach used in the past to improve the local search methods. With respect to MAXSAT, there are very few attempts in designing genetic algorithms [4, 20]. In fact, several researchers have pointed out the difficulties in designing a genetic algorithm for MAXSAT [19, 1, 28, 29]. Nevertheless, there has been successful research done over the years to use crossover operators by exploiting the structure and decomposability of SAT instances [7, 8, 9]

In this section, we will review the parallel genetic algorithms implemented on CPU, GPU and FPGA architectures.

3.1 Review of initial papers

In this section, we will review the initial papers that were given as a part of this exam and establish its relevance to the topic of discussion.

3.1.1 Adaptive memory based local search for MAXSAT

The local search methods for MAXSAT performs $\Theta(1)$ bit flip every iteration. The critical point in the construction of robust local search methods for MAXSAT is in the identification of an appropriate bit to flip. Several types of research have focused on this aspect while considering two features, namely, intensification and diversification. These two features are essential to consider to escape from the local optimum. GSAT [31] and WalkSAT [30] are two standard local search methods. GSAT chooses the best variable to flip among all the variables occurring in the unsatisfied clauses, thereby exploiting

the intensification property. However, WalkSAT first randomly selects one unsatisfied clause and then always pick a variable from the selected variable to flip, thus achieving diversification. In [22], the authors come up with a local search algorithm called Adaptive memory-based local search (AMLS) to solve the MAXSAT problem. Specifically, they make use of the history collected during the course of bit flips of the local search process. They combine the intensification and diversification methods, and take advantage of the existing techniques and introduce five new changes. With these changes, they were able to outperform AdaptG2WSAT, a state-of-the-art local search solver.

Evaluation: Like any other local search methods, AMLS also suffers from solving structured instances. The adaptive parameter adjusting mechanism uses several pre-defined parameters, and it requires manual tuning. The authors do not make a note of how much memory they store and how does that effect with an increasing number of variables and clauses. They have tested only up to 4000 variables, but in reality, the MAXSAT solvers can solve 1 million variables. The trade-off in using excess memory is not well explained in this article.

Relevance: It is not related to parallel genetic algorithm except that it performs better than the state-of-the-art local search heuristic AdaptG2SAT and hence forms a potential local search mechanism for integrating into a hybrid genetic algorithm framework.

3.1.2 Tunneling Between Plateaus: Partition Crossover

In [8], the authors point out two main short-comings of local search algorithms and develop a framework called PXSAT for solving the MAXSAT instances. In this framework, the authors combine the local search algorithm with the partition crossover operator. Specifically, they apply the partition crossover operator on two local minima obtained at t and αt if the local search doesn't find an improvement in α time steps.

Evaluation: There are two weakness or gap in the literature, as pointed out in the article. First is the running time of the PXSAT framework and the authors suggest ways to improve it. Second is the measure that they use to report when and how pXSAT works. Although it is understandable that the framework works better with more number of partitions, I believe the measure can be much improved.

Relevance: This work doesn't bear a direct correlation to the parallel genetic algorithms. However, I believe that the partition crossover is a powerful crossover operator to implement in a GA framework mainly because it transmits alleles from the parents and hence, the communication cost for the evaluation function is zero.

3.2 PGA on CPU

In [23], the authors make a comparison of four PGA models for MAXSAT. They use one instance of MAXSAT with 100 variables and 430 clauses. Although this is a *hard* problem (since the ratio of clauses to variables is 4.3), their conclusions may or may not hold for other instances. In this section, we will review one of the earliest methods [10] that study parallel genetic algorithms (PGA)

for optimization of the 3CNF problem. 3CNF is nothing but a MAX3SAT problem. The objective is to find the variable assignment that maximizes the number of satisfied clauses or minimizes the number of unsatisfied clauses. This article is from 1994 and does an excellent job of providing valuable insights into the design of PGAs. To the best of my knowledge, no other work has addressed the problem of implementing PGAs for MAXSAT on CPUs. Much of the parallelization work has been focused on improving local search methods [14].

The author chooses the island model to implement the PGA wherein, they divide the population into subpopulations, and there is a migration of individuals between the islands at the specific intervals in a ring fashion. They preserve the diversity in the population by controlling the migration rate and interval. The author constructs a simple genetic algorithm (SGA) and a hill climbing genetic algorithm (HGA) wherein they perform the hill climbing local search before applying genetic operators. The paper does not make a mention on the details of genetic operators. The chromosome has the standard encoding of 0 and 1, indicating the truth assignment of the variables. The count on the number of satisfied clauses is the evaluation function.

The author tested the GA framework on 100 randomly generated "hard" CNF problems, each containing 25 variables. They make four observations. First, they conduct a simple experiment to test the convergence on the constant and logarithmic model. In the constant model, they have a population of 40, and the time to converge is constant. In the logarithmic model, they have only four individuals, and the convergence time is logarithmic in the number of variables. They find the performance of the constant model to be superior to that of the logarithmic model and hence fix the population size and number of generations to 40 and 300 for the rest of their experiments.

Second, they conduct the compare SGA and HGA for various mutation and crossover rates and find that the GA with hill climbing local search technique is better when compared to an SGA. This observation is consistent with several methods developed in the past two decades. Next, they perform an analysis of the migration rate and intervals for the PGA versions. Finally, they conclude that parallelism using the current set up does not improve the performance of the 3CNF problems. They observed that the number of instances solved by PGA reduced by 50% when compared to the sequential version. They randomly select individuals to migrate and propose that elitism might be a better way to solve the issue.

Evaluation: This is one of the earliest approaches in the design of PGA for MAXSAT. The author does not provide a clear explanation of the details of the crossover operator. Furthermore, there are no details of the machine configuration. However, the observations made are consistent with several research conducted in the following years.

Relevance: This study is highly relevant to the design of PGAs. The authors suggest the following as future directions (1) Elitism when selecting the migrating individuals and reverse elitism when replacing the accepted migrant (2) Adaptive migration schemes instead of the constant migration (3) Other communication topologies like grids, hypercubes to transfer the chromosomes among the parallel units (4) Implementation of the PGA on other parallel machines such as distributed and

Massively parallel architectures. As a result, in 2011, the authors of [27], experiment the adaptive migration technique on MAXSAT instances on a distributed parallel genetic algorithm model. The authors conclude that the adaptive migration technique improves performance when compared to constant migration.

3.3 PGA on GPU

In [24], the authors developed a hybrid genetic algorithm with hill climbing as the local search operator. The island model of parallelization is used where each thread block acts as an island and threads in a thread block acts as the individuals in the island. Selection for mating partner happens via a process called diffusion where one of the north, south, east, west neighbors are picked using the binary tournament selection operator. The mutation operator is a simple inversion of a random gene. The recombination is done using specific masking scheme. Elitism replacement is used to insert the offspring into the population. The child replaces the parent if and only if it has a better evaluation when compared to the parent.

The proposed genetic algorithm framework is launched as two sets of the kernel on the GPUs. The first kernel is a random number generator where all the random numbers required for the GA framework are generated. Owing to the GPU hardware limitations, the random numbers are generated well in advance using this specialized kernel. The second kernel is the implementation of the GA framework.

There are two levels of optimization performed on this simple GA framework. The first pass of optimization focuses on reducing the synchronization cost by replacing all the occurrences of conditional statements with deterministic statements. With the proposed optimization, the execution time of the naive implementation (82.713 seconds) goes down to 37.76 seconds on uf250-01 MAXSAT instance.

The second level of optimization involves applying architecture specific modifications. They apply four techniques and see a further reduction in the execution time from 37.76 to 28.9 seconds on the uf250-01 MAXSAT instance. They observe similar performance gains on ten other MAXSAT instances. The GPU results are compared against four other CPU versions, as shown in Table 2 of the article. On average, a speedup of $13\times$ to $25\times$ over CPU implementations is observed.

Evaluation: This article has severe limitations on the choice of the dataset. First, the solution quality is not as good when compared to the state-of-the-art local search heuristics. Second, the maximum number of variables chosen for this study is only 250. However, this GA framework is one of the best MAXSAT implementations so far in the literature.

3.4 PGA on FPGA

The implementation of the population-based genetic algorithms on the FPGA platform has several challenges. The authors [13] address three of the problems. First, the algorithm needs to wait until all the offsprings are generated. Second, the algorithm required a large amount of memory to operate

Table 3: List of MAXSAT Genetic operators surveyed

Crossover	Mutation	Local search	Selection
PX [8]	Flip-flop [13]	AMLS [22]	Binary Tournament [24]
N-point [13]	Constrained [13]		
Blending [13]	Swap [13]		

on the entire population. Finally, there is a need to manually re-tune the algorithm parameters to escape from the local minima.

To address these problems, the authors develop something called the pipelined genetic propagation (PGP) and make the following three contributions. First, they structure the genetic algorithm as a graph of parallel pipelined components with local candidate storage, thereby breaking the data dependency and eliminating the shared memory bottleneck. Second, they implement a topology generator for producing PGP solver instances, allowing multiple candidate graphs to be created for a given FPGA. Finally, they can change the topology of the solver at run-time dynamically.

They create something called genetic nodes that form the primary component of a PGP solver. These genetic nodes are created for all the three genetic operations - selection, mutation, and crossover. Since crossover and mutation operators can change the evaluation of the individual, they are called blind nodes, whereas the selection node is termed as a non-blind node. They construct something called the *topology mapping unit* to manage the connection of these nodes. The PGP scheme has a substantial advantage over generational genetic algorithm in terms of pipeline efficiency. Since the nodes are pipelined and connected logically, the hardware pipeline stalls are efficiently managed by this method.

They evaluate on both the TSP and MAXSAT instances. For TSP, they consider a 64 city problem. The specific MAXSAT instance is not explained. Overall, they compare the FPGA resource utilization and conclude that the PGP system outperforms other FPGA solvers in terms of optimization power and computational efficiency.

Evaluation: It is an interesting research article and has several potentials for future design. Comparing it with the state-of-the-art CPU techniques is not justifiable, and I believe the paper is self-sufficient and well written.

3.5 Summary

To summarize the PGA methods for MAXSAT solvers, none of the genetic algorithm methods can outperform the local search methods. The FPGA based articles do an excellent job of carefully describing the challenges and incrementally address them. Even then, there is a considerable potential to improve the techniques to handle large problem instances. Table 3 lists the genetic operators surveyed in this article. Potential future directions are,

- **Design of parallel genetic algorithms:** The future genetic operators or the enhancements

to the existing methods must incorporate the parallel design choices.

- **Creative incorporation of local search data structures for PGAs:** Since the LS methods are unbeatable; it is highly essential to design PGAs that can creatively take advantage of them.
- **Usage of alternate parallel framework:** Rather than thinking the parallelization of PGAs as across the population, future designs can be crafted to use an alternate parallel framework. Some examples are provided in Section 4.

4 Alternative Parallel Framework

In this section, we discuss some alternative and recent advances in the design of parallel heuristics that may be relevant to the design of PGAs. To ease the burden of designing parallel search methods, researchers resort to so-called portfolio method of parallelization. Here, they run several sequential solvers in parallel. A brief survey on one of the classic portfolio method for SAT is discussed in 4.2. Section 4.1 presents guidelines for the design and implementation of effective hybrid metaheuristics using heterogeneous (CPU and GPU) resources.

4.1 Hybrid Metaheuristics on heterogeneous system

In [33], the authors provide guidelines for designing hybrid metaheuristics on multi-core CPU and single GPU systems. They develop three different parallel models. The first model is called the solution-level parallel model, wherein the focus is on the parallel evaluation of a single solution. These models apply to parallel genetic operators and evaluation functions. The second model is the iteration level parallel model, wherein the evaluation of the solutions are performed in a model similar to that of the master-slave model. The task partitioning in either of these models is such that the entire sequential part must be handled by the CPU system and the data-intensive parts be sent to the GPU.

The third model is where the parallelization is explored at the algorithmic level. Here several metaheuristics are run in parallel. This model can also be termed as the portfolio method and is described in Section 4.2. Since each parallel unit runs its algorithm, they may not share the same piece of code and instructions. As a result, these type of parallel models are suited for CPU architecture and not for SIMD machines. Finally, they provide guidelines on how to efficiently balance the load between CPU and GPU.

Evaluation: They evaluate their models on Fast Ant System and Quadratic Assignment Problem. Since these are not relevant to the MAXSAT and TSP, details are not provided.

Relevance: These guidelines are relevant when designing PGAs for MAXSAT and TSP solvers.

4.2 Portfolio method

As mentioned in the previous section, to escape from the pain of designing parallel genetic operators, one alternative is to run different solvers in parallel and share useful information among them. One other alternative is to run the same solver starting from a different set of the population, thereby speeding up the rate of convergence. In this article, [21], the authors develop a parallel portfolio selection method for SAT solvers. They talk about Clause sharing as a means to reduce the redundant work in parallel SAT solving. Since this article is not focused on genetic algorithms, I do not summarize and evaluate. However, the parallel portfolio method is one promising alternative to think of implementing PGAs.

5 Conclusion

Genetic algorithms are embarrassingly parallel and are very challenging to implement on parallel architectures. In the design of PGAs, there are five primary criteria to be considered namely, the memory, communication cost, solution quality, architecture resource utilization, and the parameter tuning of the appropriate PGA model such as the migration rate and intervals. In this article, a brief survey on the parallel genetic algorithms for CPU, GPU, and FPGA platform has been presented.

EAX stands out as the single best PGA for TSP on the CPU platform. There are very few attempts concerning FPGAs. With regards to GPUs, one specific approach based on ARX and BCSCX crossover claim to find the global optima on 100,000 city problem. There have been minimal attempts in the design of PGAs for MAXSAT problems, and none of them can outperform the local search methods. Alternative techniques to the standard parallel genetic algorithm such as heterogeneous hybrid metaheuristics and parallel portfolios are some promising directions to pursue to ease the pain of designing PGAs. Overall, the design of parallel is essential, especially with the proliferation of heterogeneous and exascale computing technologies.

Some of the open challenges are to design parallel genetic operators to expose fine-grain parallelism. Design of communication effective evaluation function and the lightweight crossover operators are also some of the criteria to be considered. Furthermore, since MAXSAT local search solvers are unbeatable, there is a need to creatively incorporate LS data structures while designing PGAs. Alternatively, a portfolio-based parallel framework with careful sharing of information amongst the parallel units can also be used to solve PGAs effectively.

References

- [1] BERTONI, A., CARPENTIERI, M., CAMPADELLI, P., AND GROSSI, G. A genetic model: Analysis and application to maxsat. *Evolutionary Computation* 8, 3 (2000), 291–309.

- [2] BISCHL, B., KERSCHKE, P., KOTTHOFF, L., LINDAUER, M., MALITSKY, Y., FRCHETTE, A., HOOS, H., HUTTER, F., LEYTON-BROWN, K., TIERNEY, K., AND VANSCHOREN, J. Aslib: A benchmark library for algorithm selection. *Artificial Intelligence* 237 (2016), 41 – 58.
- [3] BOESE, K. D., KAHNG, A. B., AND MUDDU, S. A new adaptive multi-start technique for combinatorial global optimizations. *Operations Research Letters* 16, 2 (1994), 101 – 113.
- [4] BOUGHACI, D., BENHAMOU, B., AND DRIAS, H. Scatter search and genetic algorithms for max-sat problems. *Journal of Mathematical Modelling and Algorithms* 7, 2 (Jun 2008), 101–124.
- [5] BURTSCHER, M., NASRE, R., AND PINGALI, K. A quantitative study of irregular programs on gpus. In *2012 IEEE International Symposium on Workload Characterization (IISWC)* (Nov 2012), pp. 141–151.
- [6] CANT-PAZ, E. A survey of parallel genetic algorithms. *CALCULATEURS PARALLELES* 10 (1998).
- [7] CHEN, W., AND WHITLEY, D. Decomposing sat instances with pseudo backbones. In *Evolutionary Computation in Combinatorial Optimization* (Cham, 2017), B. Hu and M. López-Ibáñez, Eds., Springer International Publishing, pp. 75–90.
- [8] CHEN, W., WHITLEY, D., TINÓS, R., AND CHICANO, F. Tunneling between plateaus: Improving on a state-of-the-art maxsat solver using partition crossover. In *Proceedings of the Genetic and Evolutionary Computation Conference* (New York, NY, USA, 2018), GECCO '18, ACM, pp. 921–928.
- [9] CHICANO, F., OCHOA, G., WHITLEY, D., AND TINÓS, R. Enhancing partition crossover with articulation points analysis. In *Proceedings of the Genetic and Evolutionary Computation Conference* (New York, NY, USA, 2018), GECCO '18, ACM, pp. 269–276.
- [10] FRANK, J. A study of genetic algorithms to find approximate solutions to hard 3cnf problems.
- [11] FUJIMOTO, N., AND TSUTSUI, S. A highly-parallel tsp solver for a gpu computing platform. In *Numerical Methods and Applications* (Berlin, Heidelberg, 2011), I. Dimov, S. Dimova, and N. Kolkovska, Eds., Springer Berlin Heidelberg, pp. 264–271.
- [12] GUO, L., FUNIE, A. I., THOMAS, D. B., FU, H., AND LUK, W. Parallel genetic algorithms on multiple FPGAs. *SIGARCH Comput. Archit. News* 43, 4 (Apr. 2016), 86–93.
- [13] GUO, L., GUO, C., THOMAS, D. B., AND LUK, W. Pipelined genetic propagation. In *2015 IEEE 23rd Annual International Symposium on Field-Programmable Custom Computing Machines* (May 2015), pp. 103–110.
- [14] HAMADI, Y., AND SAIS, L. *Handbook of Parallel Constraint Reasoning*, 1st ed. Springer Publishing Company, Incorporated, 2018.

- [15] HELSGAUN, K. An effective implementation of the linkernighan traveling salesman heuristic. *European Journal of Operational Research* 126, 1 (2000), 106 – 130.
- [16] HONDA, K., NAGATA, Y., AND ONO, I. A parallel genetic algorithm with edge assembly crossover for 100,000-city scale TSPs. In *2013 IEEE Congress on Evolutionary Computation* (June 2013), pp. 1278–1285.
- [17] KANG, S., KIM, S.-S., WON, J., AND KANG, Y.-M. Gpu-based parallel genetic approach to large-scale travelling salesman problem. *The Journal of Supercomputing* 72, 11 (Nov 2016), 4399–4414.
- [18] KERSCHKE, P., KOTTHOFF, L., BOSSEK, J., HOOS, H. H., AND TRAUTMANN, H. Leveraging tsp solver complementarity through machine learning. *Evolutionary Computation* 26, 4 (2018), 597–620.
- [19] KILANI, Y. Comparing the performance of the genetic and local search algorithms for solving the satisfiability problems. *Applied Soft Computing* 10, 1 (2010), 198 – 207.
- [20] LARDEUX, F., SAUBION, F., AND HAO, J. Gasat: A genetic local search algorithm for the satisfiability problem. *Evolutionary Computation* 14, 2 (June 2006), 223–253.
- [21] LINDAUER, M., HOOS, H., HUTTER, F., AND LEYTON-BROWN, K. *Selection and Configuration of Parallel Portfolios*. Springer International Publishing, Cham, 2018, pp. 583–615.
- [22] LÜ, Z., AND HAO, J.-K. Adaptive memory-based local search for max-sat. *Appl. Soft Comput.* 12, 8 (Aug. 2012), 2063–2071.
- [23] LUQUE, G., AND ALBA, E. *Parallel Genetic Algorithms: Theory and Real World Applications*. Springer Publishing Company, Incorporated, 2013.
- [24] MUNAWAR, A., WAHIB, M., MUNETOMO, M., AND AKAMA, K. Hybrid of genetic algorithm and local search to solve MAX-SAT problem using nVidia CUDA framework. *Genetic Programming and Evolvable Machines* 10, 4 (Oct 2009), 391.
- [25] NAGATA, Y., AND KOBAYASHI, S. A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. *INFORMS J. on Computing* 25, 2 (Apr. 2013), 346–363.
- [26] OCHOA, G., AND VEERAPEN, N. Mapping the global structure of TSP fitness landscapes. *Journal of Heuristics* 24, 3 (Jun 2018), 265–294.
- [27] OSORIO, K., LUQUE, G., AND ALBA, E. Distributed evolutionary algorithms with adaptive migration period. In *2011 11th International Conference on Intelligent Systems Design and Applications* (Nov 2011), pp. 259–264.

- [28] RANA, S., HECKENDORN, R. B., AND WHITLEY, D. A tractable walsh analysis of sat and its implications for genetic algorithms. In *Proceedings of the Fifteenth National/Tenth Conference on Artificial Intelligence/Innovative Applications of Artificial Intelligence* (Menlo Park, CA, USA, 1998), AAAI '98/IAAI '98, American Association for Artificial Intelligence, pp. 392–397.
- [29] RANA, S. B., AND WHITLEY, L. D. Genetic algorithm behavior in the maxsat domain. In *Proceedings of the 5th International Conference on Parallel Problem Solving from Nature* (London, UK, UK, 1998), PPSN V, Springer-Verlag, pp. 785–794.
- [30] SELMAN, B., KAUTZ, H. A., AND COHEN, B. Noise strategies for improving local search. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (Vol. 1)* (Menlo Park, CA, USA, 1994), AAAI '94, American Association for Artificial Intelligence, pp. 337–343.
- [31] SELMAN, B., LEVESQUE, H., AND MITCHELL, D. A new method for solving hard satisfiability problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence* (1992), AAAI'92, AAAI Press, pp. 440–446.
- [32] TOMASSINI, M. *Spatially Structured Evolutionary Algorithms: Artificial Evolution in Space and Time (Natural Computing Series)*. Springer-Verlag, Berlin, Heidelberg, 2005.
- [33] VAN LUONG, T., TAILLARD, E., MELAB, N., AND TALBI, E.-G. Parallelization strategies for hybrid metaheuristics using a single gpu and multi-core resources. In *Parallel Problem Solving from Nature - PPSN XII* (Berlin, Heidelberg, 2012), C. A. C. Coello, V. Cutello, K. Deb, S. Forrest, G. Nicosia, and M. Pavone, Eds., Springer Berlin Heidelberg, pp. 368–377.
- [34] XIE, X., AND LIU, J. Multiagent optimization system for solving the traveling salesman problem (tsp). *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 39, 2 (April 2009), 489–502.
- [35] YU, Q., CHEN, C., AND PAN, Z. Parallel genetic algorithms on programmable graphics hardware. In *Advances in Natural Computation* (Berlin, Heidelberg, 2005), L. Wang, K. Chen, and Y. S. Ong, Eds., Springer Berlin Heidelberg, pp. 1051–1059.
- [36] ZHANG, K., YANG, S., LI, L., AND QIU, M. Parallel genetic algorithm with opencl for traveling salesman problem. In *Bio-Inspired Computing - Theories and Applications* (Berlin, Heidelberg, 2014), L. Pan, G. Păun, M. J. Pérez-Jiménez, and T. Song, Eds., Springer Berlin Heidelberg, pp. 585–590.